



New Mansoura University
Faculty of Computer Science and Engineering



Advanced Machine Learning

Clustering

Islam Reda

Associate Professor

islam-cis@mans.edu.eg



Outline

- Clustering
- K-Mean
 - The K-Means Algorithm
 - Centroid Initialization Methods
 - Finding the Optimal Number of Clusters
 - Limits of K-Means



Unsupervised Learning Techniques

- Although most of the applications of Machine Learning today are based on supervised learning, the vast majority of the available data is actually unlabeled.
- Labeling data is a **long, costly, and tedious** task.
- It would be great if the algorithm could just exploit the unlabeled data without needing to label it (**unsupervised learning**).



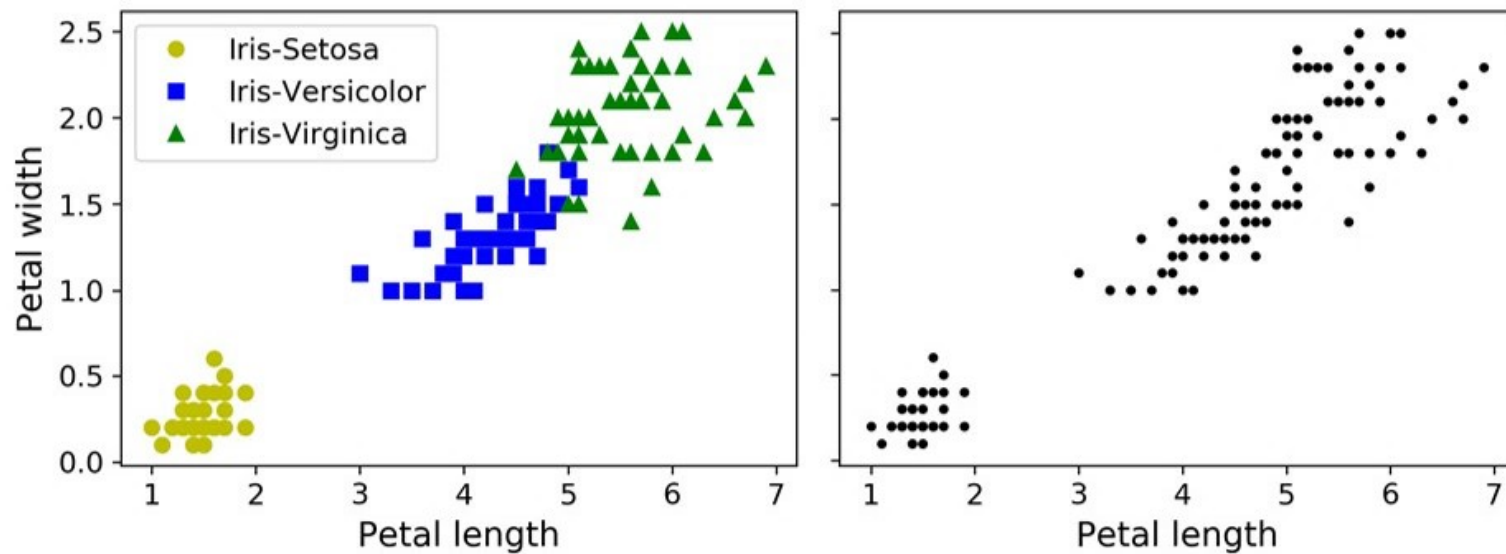
Unsupervised Learning Techniques

The most common unsupervised learning tasks:

- **Clustering:** the goal is to group similar instances together into clusters.
- **Anomaly detection:** the objective is to learn what “normal” data looks like, and use this to detect abnormal instances.
- **Density estimation:** this is the task of estimating the probability density function (PDF) of the random process that generated the dataset.

Clustering

- **Clustering:** it is the task of identifying similar instances and assigning them to **clusters**, i.e., groups of similar instances.



Classification (left) versus clustering (right)



Clustering

Clustering is used in a wide variety of applications, including:

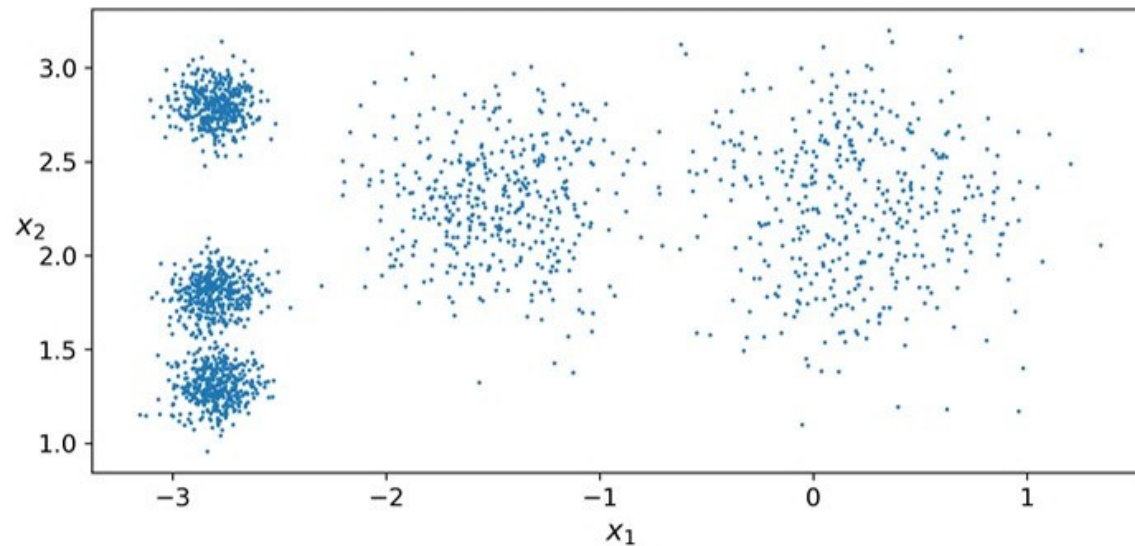
- **For customer segmentation:** You can adapt your products and marketing campaigns to each segment.
- **As a dimensionality reduction technique:** once a dataset has been clustered, it is usually possible to measure each instance's **affinity** with each cluster (affinity is any measure of how well an instance fits into a cluster).
- **For anomaly detection (outlier detection):** any instance that has a low affinity to all the clusters is likely to be an anomaly.



Clustering

- **For search engines:** some search engines let you search for images that are similar to a reference image.
- **To segment an image:** by clustering pixels according to their color, then replacing each pixel's color with the mean color of its cluster

K-Means



An unlabeled dataset composed of five blobs of instances

- The K-Means algorithm is a simple algorithm capable of clustering this kind of dataset very quickly and efficiently.



K-Means

- Let's train a K-Means clusterer on this dataset.

```
from sklearn.cluster import KMeans  
k = 5  
kmeans = KMeans(n_clusters=k)  
y_pred = kmeans.fit_predict(X)
```

- Note that you have to specify the number of clusters k that the algorithm must find. In this example, it is pretty obvious, but in general it is not that easy.
- Each instance was assigned to one of the 5 clusters.



K-Means

- The KMeans instance preserves a copy of the labels of the instances it was trained on, available via the **labels_** instance variable:

```
>>> y_pred
array([4, 0, 1, ..., 2, 1, 0], dtype=int32)
>>> y_pred is kmeans.labels_
True
```

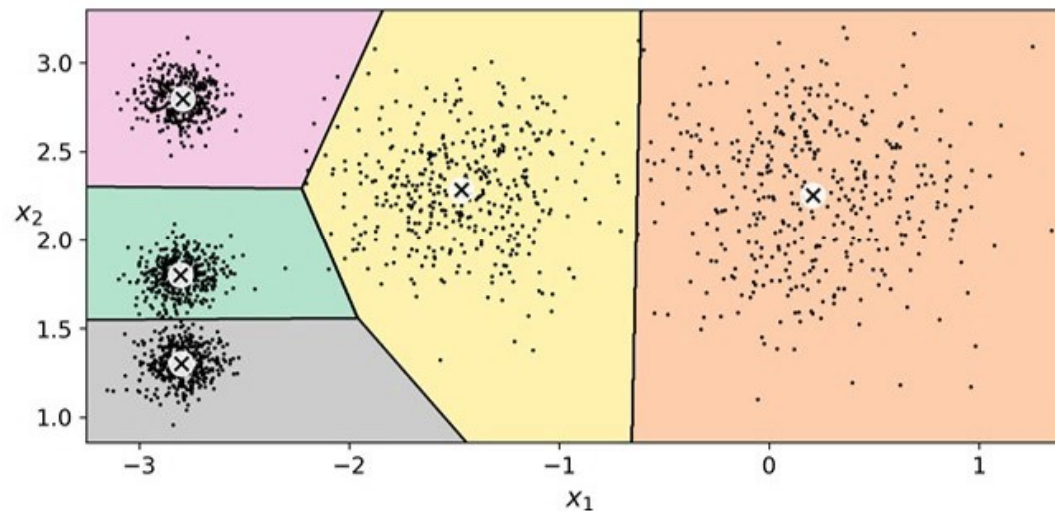
- We can also take a look at the 5 centroids that the algorithm found:

```
>>> kmeans.cluster_centers_
array([[ -2.80389616,  1.80117999],
       [ 0.20876306,  2.25551336],
       [-2.79290307,  2.79641063],
       [-1.46679593,  2.28585348],
       [-2.80037642,  1.30082566]])
```

K-Means

- You can assign new instances to the cluster whose centroid is closest:

```
>>> X_new = np.array([[0, 2], [3, 2], [-3, 3], [-3, 2.5]])  
>>> kmeans.predict(X_new)  
array([1, 1, 2, 2], dtype=int32)
```



K-Means decision boundaries (Voronoi tessellation)



K-Means

- **Hard clustering** is assigning each instance to a single cluster.
- **Soft clustering** is giving each instance a score per cluster. For example, the score can be the distance between the instance and the centroid, or it can be a similarity score (or affinity).
- In the KMeans class, the **transform()** method measures the distance from each instance to every centroid:

```
>>> kmeans.transform(X_new)
array([[2.81093633, 0.32995317, 2.9042344 , 1.49439034, 2.88633901],
       [5.80730058, 2.80290755, 5.84739223, 4.4759332 , 5.84236351],
       [1.21475352, 3.29399768, 0.29040966, 1.69136631, 1.71086031],
       [0.72581411, 3.21806371, 0.36159148, 1.54808703, 1.21567622]])
```

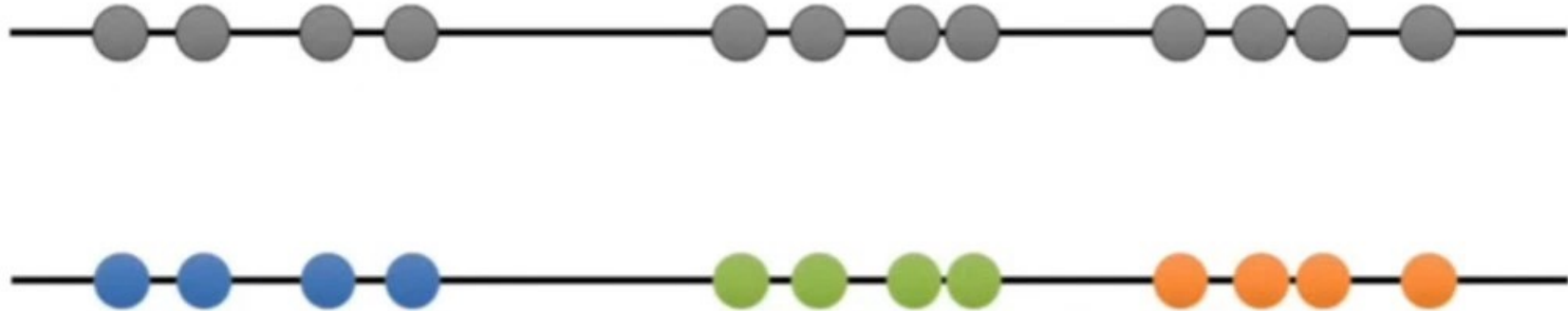


The K-Means Algorithm

- **Initialization:** We begin by randomly selecting k cluster centroids.
- **Assignment Step:** Each data point is assigned to the nearest centroid, forming clusters.
- **Update Step:** After the assignment, we recalculate the centroid of each cluster by averaging the points within it.
- **Repeat:** This process repeats until the centroids no longer change or the maximum number of iterations is reached.



The K-Means Algorithm





The K-Means Algorithm



Step 1: Select the number of clusters you want to identify in your data.

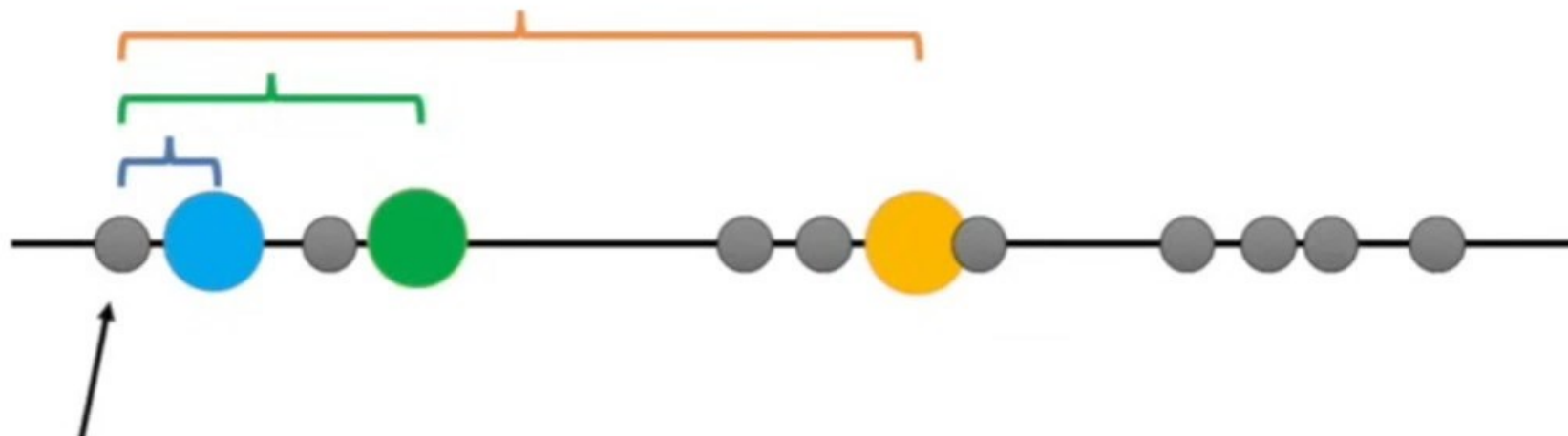
This is the “K” in “K-means clustering”. In this case, we'll select $K=3$.

Step 2: Randomly select 3 distinct data points.



The K-Means Algorithm

Step 3: Measure the distance between the 1st point and the three initial clusters.



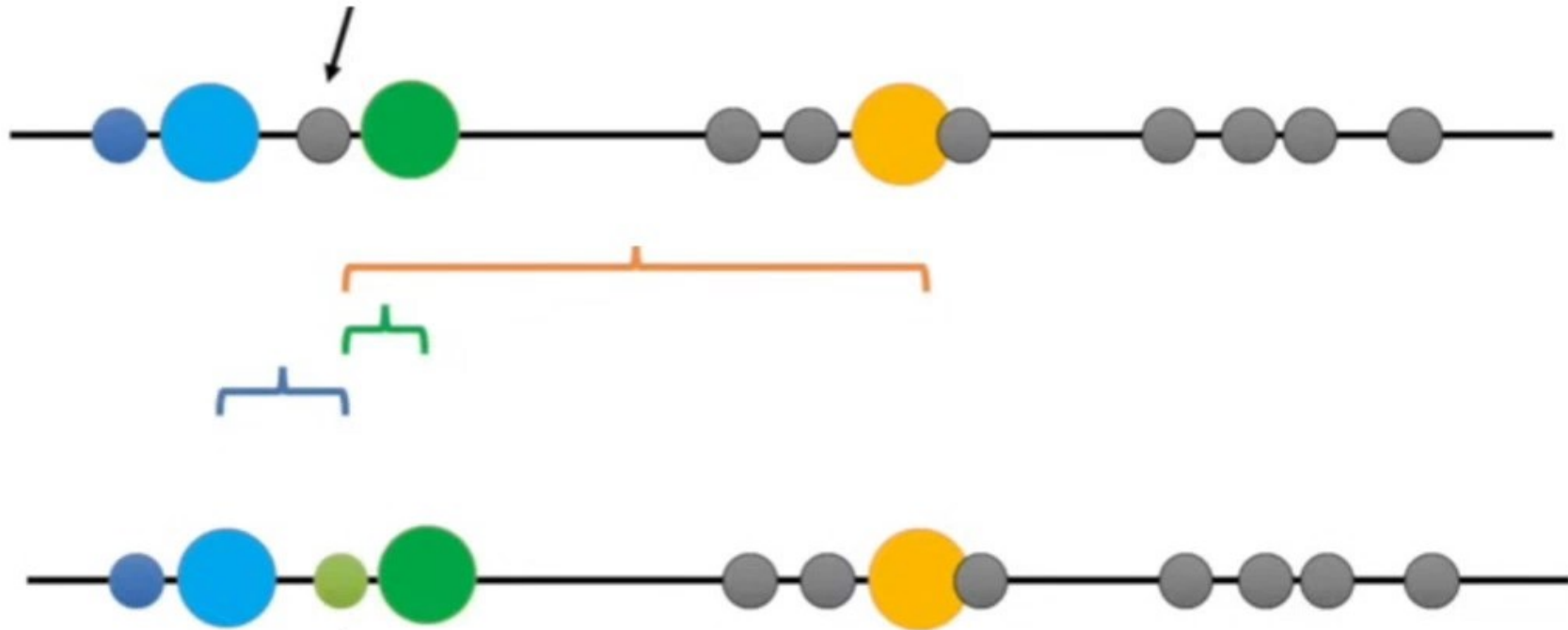
Step 4: Assign the 1st point to the nearest cluster. In this case, the nearest cluster is the blue cluster.





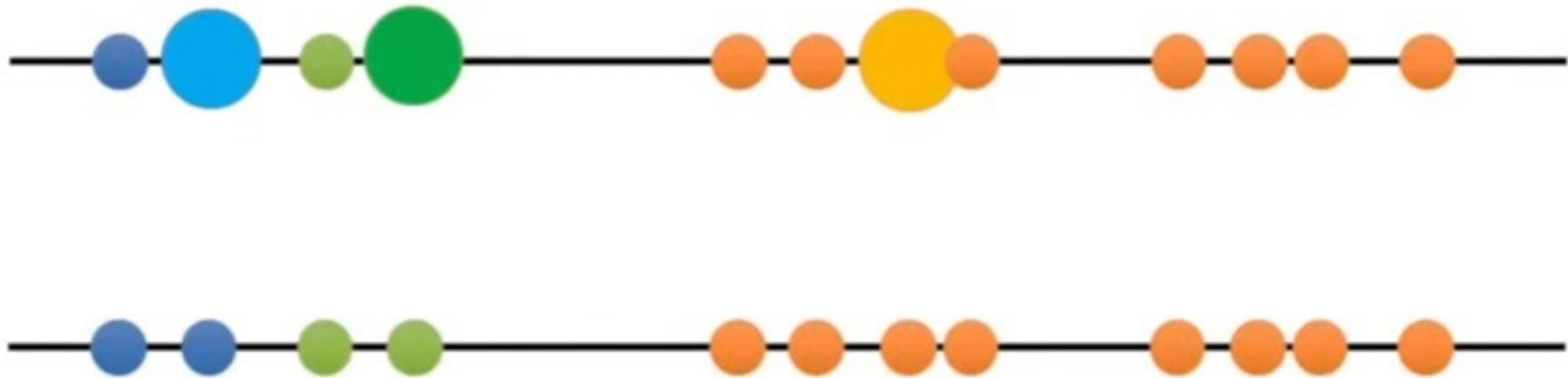
The K-Means Algorithm

Now do the same thing for the next point.



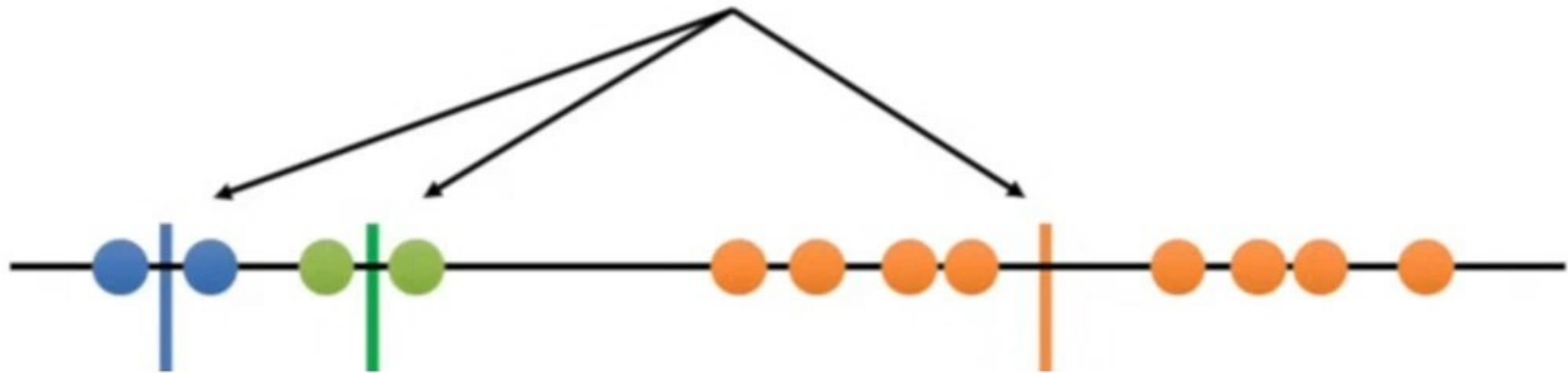


The K-Means Algorithm



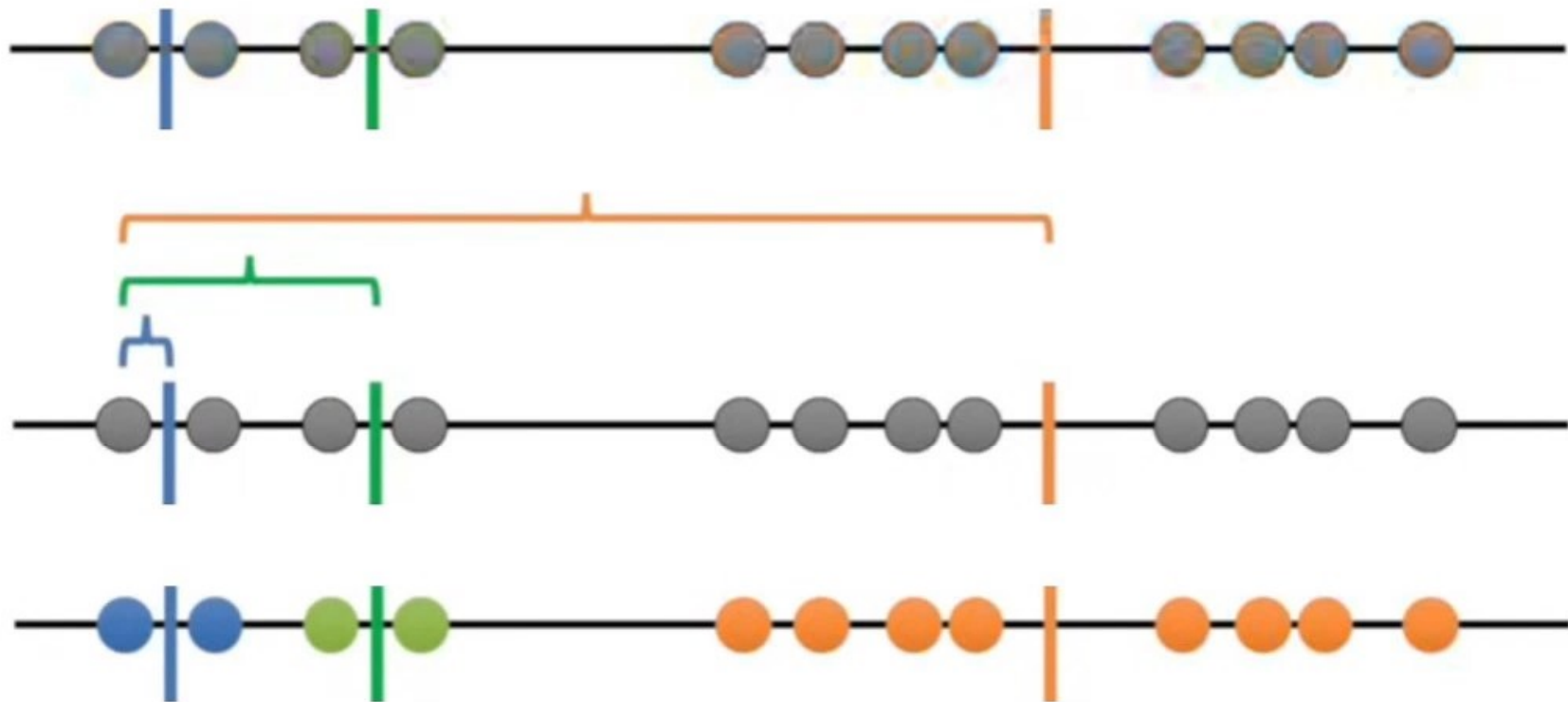
The K-Means Algorithm

Step 5: Calculate the mean of each cluster.



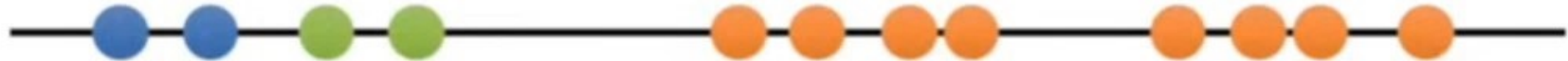


The K-Means Algorithm





The K-Means Algorithm

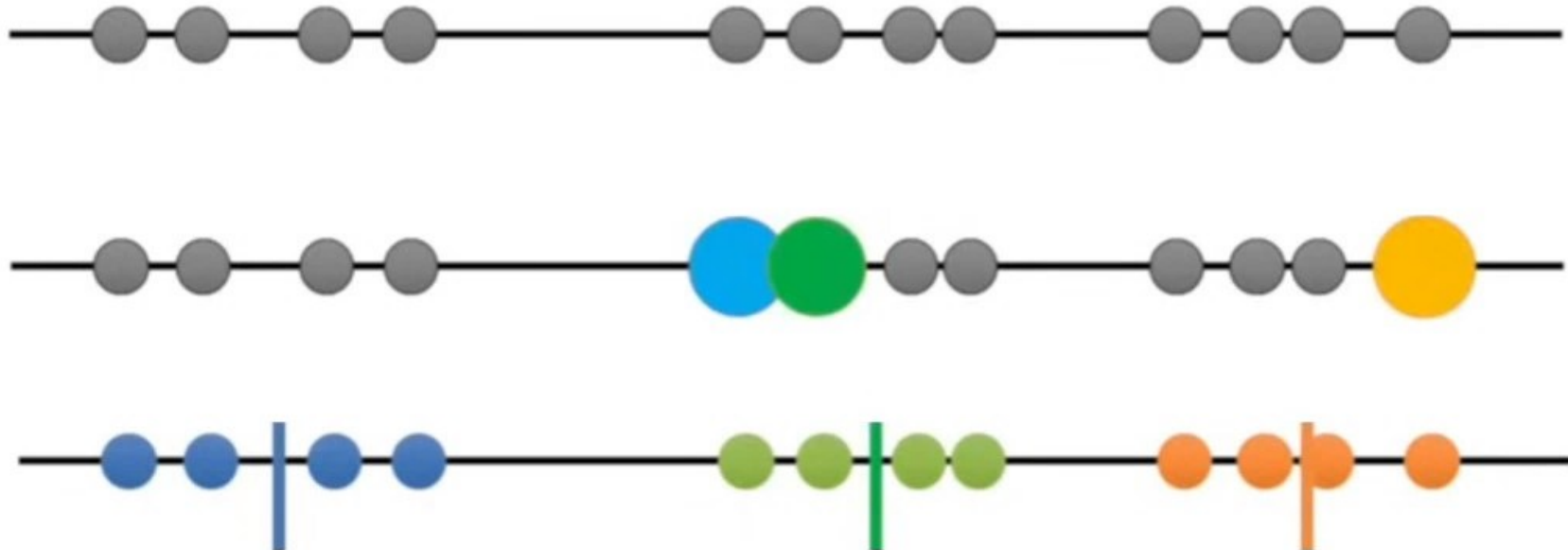


The k-means clustering is pretty terrible compared to what we did by eye.

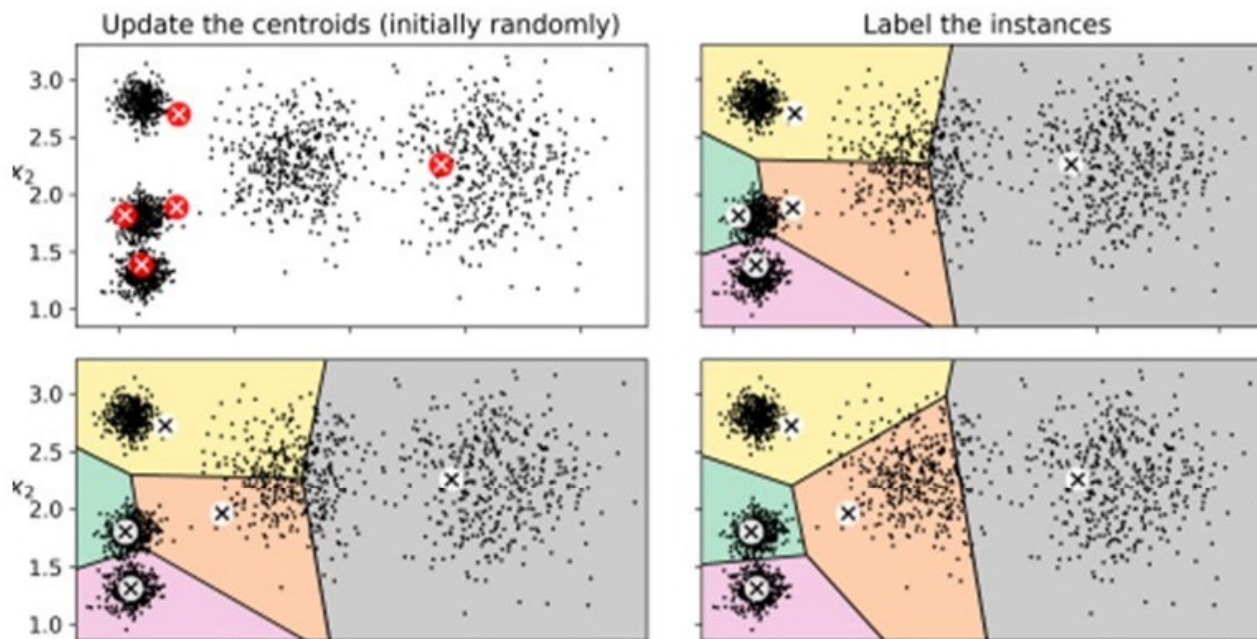




The K-Means Algorithm



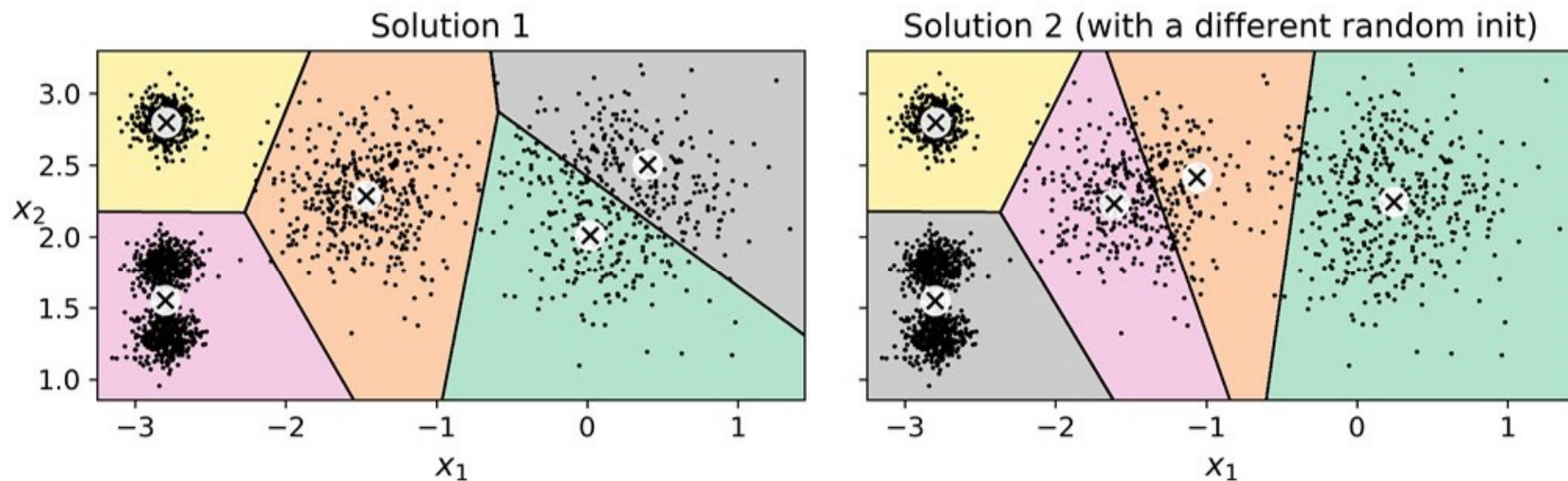
The K-Means Algorithm



The K-Means algorithm

The K-Means Algorithm

- The algorithm is **guaranteed to converge** in a finite number of steps. But, it may not converge to the right solution (i.e., it **may converge to a local optimum**): this **depends on the centroid initialization**.



Sub-optimal solutions due to unlucky centroid initializations



Centroid Initialization Methods

- If you happen to know approximately where the centroids should be, then you can set the **init** hyperparameter to a NumPy array containing the list of centroids, and set **n_init** to 1:

```
good_init = np.array([[ -3, 3], [ -3, 2], [ -3, 1], [ -1, 2], [ 0, 2]])  
kmeans = KMeans(n_clusters=5, init=good_init, n_init=1)
```

- Another solution is to **run the algorithm multiple times** with different random initializations and **keep the best solution**. This is controlled by the **n_init** hyperparameter: by default, it is equal to 10.
- The **best solution** is the model **with the lowest model's inertia**: this is the mean squared distance between each instance and its closest centroid.



Centroid Initialization Methods

- A model's inertia is accessible via the **inertia_** instance variable:

```
>>> kmeans.inertia_  
211.59853725816856
```

- The `score()` method returns the negative inertia.

```
>>> kmeans.score(X)  
-211.59853725816856
```

- **K-Means++** uses a smarter initialization step that tends to **select centroids that are distant from one another**, and this makes the K-Means algorithm much less likely to converge to a suboptimal solution.



Mini-batch K-Means

- Another important variant of the K-Means algorithm is **mini-batch K-Means**.
- Instead of using the full dataset at each iteration, the algorithm is capable of using mini-batches, moving the centroids just slightly at each iteration. This **speeds up the algorithm** typically **by a factor of 3 or 4** and makes it possible to **cluster huge datasets that do not fit in memory**.

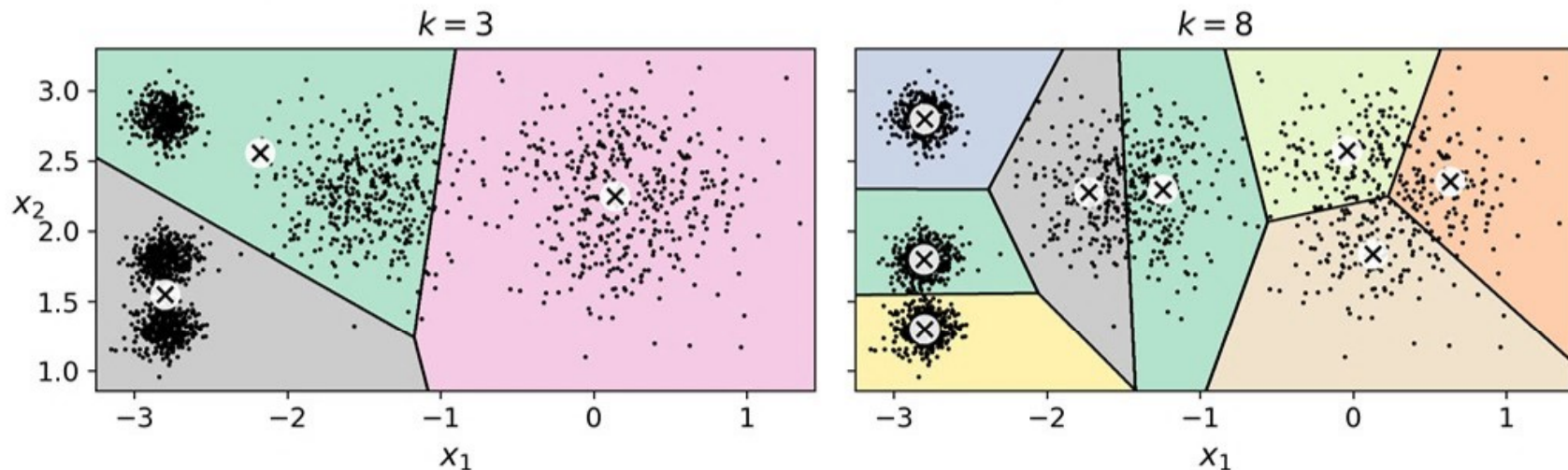
```
from sklearn.cluster import MiniBatchKMeans
```

```
minibatch_kmeans = MiniBatchKMeans(n_clusters=5)  
minibatch_kmeans.fit(X)
```

- Faster but its inertia is generally slightly worse.

Finding the Optimal Number of Clusters

- In general, it will not be so easy to know how to set k , and the result might be quite bad if you set it to the wrong value.

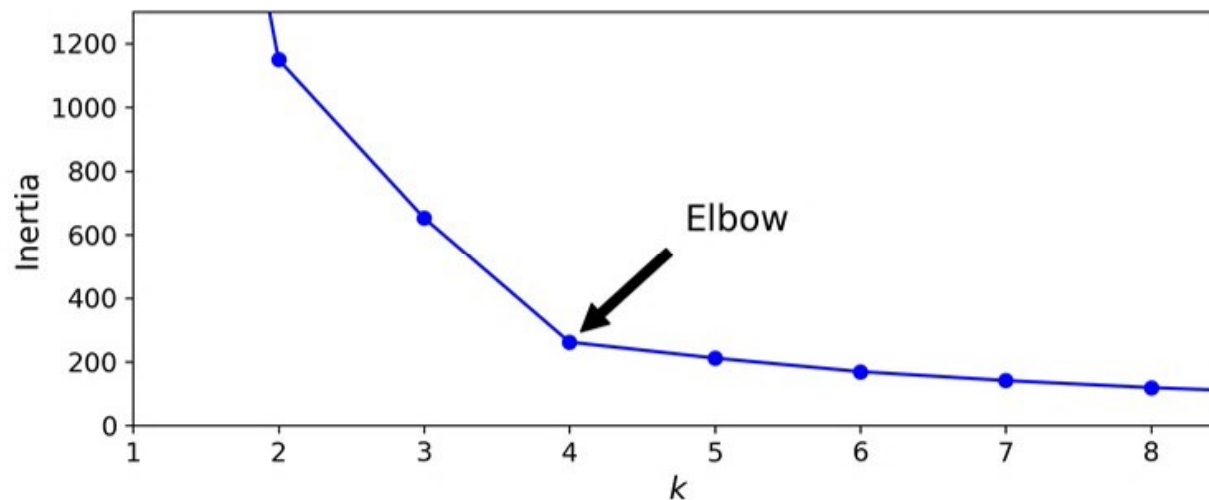


Bad choices for the number of clusters



Finding the Optimal Number of Clusters

- You might be thinking that we could just pick the model with the lowest inertia.
- The inertia keeps getting lower as we increase k



Selecting the number of clusters k using the “elbow rule”



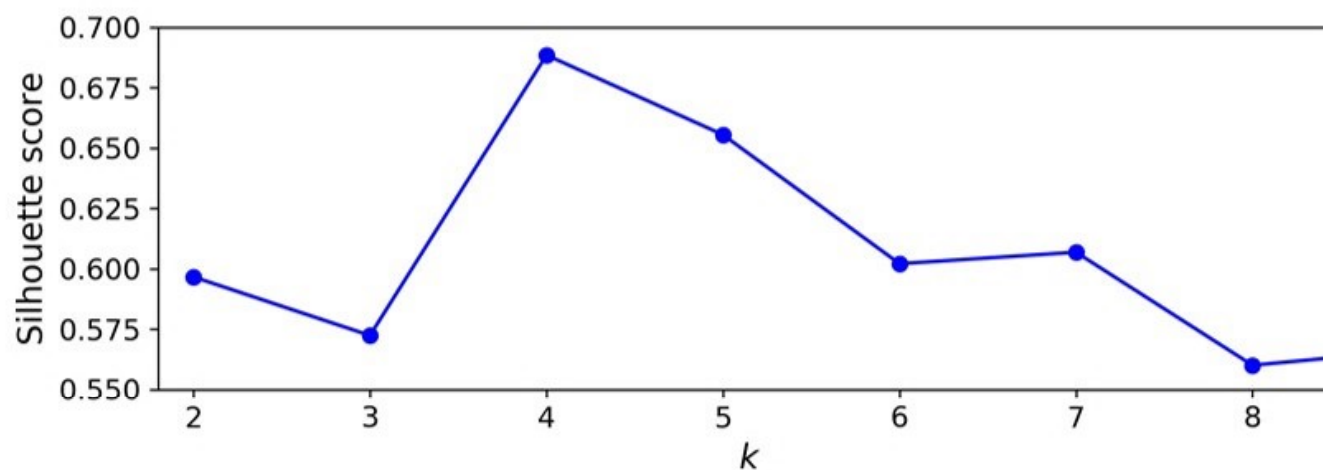
Finding the Optimal Number of Clusters

- A more precise approach (but also more computationally expensive) is to use the **silhouette score**, which is the mean silhouette coefficient over all the instances.
- The silhouette coefficient can vary between -1 and +1
- To compute the silhouette score, you can use Scikit-Learn's **silhouette_score()** function, giving it all the instances in the dataset, and the labels they were assigned:

```
>>> from sklearn.metrics import silhouette_score
>>> silhouette_score(X, kmeans.labels_)
0.655517642572828
```


Finding the Optimal Number of Clusters

- Let's compare the silhouette scores for different numbers of clusters:



Selecting the number of clusters k using the silhouette score



Limits of K-Means

- It is necessary to **run the algorithm several times** to avoid sub-optimal solutions.
- You need to **specify the number of clusters**, which can be quite a hassle.
- K-Means **does not behave very well when the clusters have varying sizes**, different densities, or non-spherical shapes.
- It is important to **scale the input features** before you run K-Means.



Thank You